

Unit – 7 Function and Packages [5 Hrs.]

PL/SQL Function

The PL/SQL Function is very similar to PL/SQL Procedure. **The main difference between procedure and a function is, a function must always return a value, and on the other hand a procedure may or may not return a value.** Except this, all the other things of PL/SQL procedure are true for PL/SQL function too.

Creating a Function

A standalone function is created using the **CREATE FUNCTION** statement. The simplified syntax for the **CREATE OR REPLACE PROCEDURE** statement is as follows –

```
CREATE [OR REPLACE] FUNCTION function_name
(parameter_list)
RETURN return_datatype
{IS | AS}
BEGIN
    < function_body >
END [function_name];
```

Where,

- **function-name** specifies the name of the function.
- **[OR REPLACE]** option allows the modification of an existing function.
- **The optional parameter list contains name, mode and types of the parameters**
- **The function must contain a return statement.**
- The **RETURN** clause specifies the data type you are going to return from the function.
- **function-body** contains the executable part.
- **The AS keyword is used instead of the IS keyword for creating a standalone function.**

Example of creating and invoking a function:

```
CREATE OR REPLACE FUNCTION myfunction(
    num1 IN NUMBER,
    num2 IN NUMBER
)
RETURN NUMBER
IS
    res NUMBER;
BEGIN
    res:=num1+num2;
    RETURN res;
END;
```

Invoking above function,

```
DECLARE
    res NUMBER;
BEGIN
    res:=myfunction(20,30);
    DBMS_OUTPUT.PUT_LINE('Result: ' || res);
END;
```

Output:

Statement processed.
Result: 50

Example – 2

The following example demonstrates Declaring, Defining, and Invoking a Simple PL/SQL Function that computes and returns the maximum of two values.

```
DECLARE
    a number;
    b number;
    c number;
FUNCTION findMax(x IN number, y IN number)
RETURN number
IS
    z number;
BEGIN
    IF x > y THEN
        z:= x;
    ELSE
        z:= y;
    END IF;
    RETURN z;
END;

-- Invoking a function
BEGIN
    a:= 23;
    b:= 45;
    c := findMax(a, b);
    dbms_output.put_line(' Maximum of (23,45): ' || c);
END;
/
```

Output:

Maximum of (23,45): 45
PL/SQL procedure successfully completed.

PL/SQL Recursive Functions

```
DECLARE
    num number;
    factorial number;

FUNCTION fact(x number)
RETURN number
IS
    f number;
BEGIN
    IF x=0 THEN
        f := 1;
    ELSE
        f := x * fact(x-1);
    END IF;
RETURN f;
END;

--Invoking a function
BEGIN
    num:= 6;
    factorial := fact(num);
    dbms_output.put_line(' Factorial ' || num || ' is ' || factorial);
END;
/
```

Output:

Factorial 6 is 720

PL/SQL Function Example Using Table

Let's take a employee table. This example illustrates creating and calling a standalone function. This function will return the total number of **Employees** in the employee table.

Table: employee			
Id	Name	Department	Salary
1	Ram	IT	35000
2	Gita	Finance	45000
3	Hari	IT	35000
4	Sita	HR	44000

Creating a Function

```
CREATE OR REPLACE FUNCTION totalEmployees
RETURN number IS
    total number(2) := 0;
BEGIN
    SELECT count(*) into total
    FROM employee;
    RETURN total;
END;
/
```

Invoking a Function

```
DECLARE
    res number(2);
BEGIN
    res := totalEmployees();
    dbms_output.put_line('Total no. of Employees: ' || res);
END;
/
```

Output:

Total no. of Employees: 4

Dropping a Function

If you want to remove your created function from the database, you should use the following syntax.

```
DROP FUNCTION function_name;
```

Difference between Function and Procedure

SN	Functions	Procedures
1	A function has a return type and returns a value.	A procedure does not have a return type. But it returns values using the OUT parameters.
2	You cannot use a function with Data Manipulation queries. Only Select queries are allowed in functions.	You can use DML queries such as insert, update, select etc... with procedures.
3	A function does not allow output parameters	A procedure allows both input and output parameters.
4	You cannot manage transactions inside a function.	You can manage transactions inside a procedure.
5	You cannot call stored procedures from a function	You can call a function from a stored procedure.
6	You can call a function using a select statement.	You cannot call a procedure using select statements.

Optimizing Function in Execution

To optimize a function, specify the **DETERMINISTIC** keyword when creating the function to indicate that the function will always return the same result for a given input.

The following example demonstrates a function that does so by specifying the **DETERMINISTIC** keyword:

Consider the following table Employee,

EID	NAME	SALARY
101	Ram	25000
102	Shyam	55000
103	Gita	34000

```
CREATE OR REPLACE FUNCTION myfunction(emp_id IN INT)
RETURN VARCHAR2

DETERMINISTIC IS
    ename employee.name%TYPE;
BEGIN
    SELECT name INTO ename FROM employee
    WHERE eid=emp_id;
    RETURN ename;
END;
/
```

Invoking a function

```
DECLARE
    res VARCHAR2(30);
BEGIN
    res:=myfunction(102);
    DBMS_OUTPUT.PUT_LINE('Employee Name: ' || res);
END;
```

Output:

Employee Name: Shyam

How It Works

A deterministic function is one that always returns the same resulting value as long as the parameters that are passed in are the same. This type of function can be useful for improving performance.

The function will be executed only once for any given set of parameters. **This means that if the same parameters are passed to this function in subsequent calls, then the function will be bypassed and return the cached value from the last execution using those parameters.** This can really help in cases where calculations are being performed and repeated calls to the function may take a toll on performance.

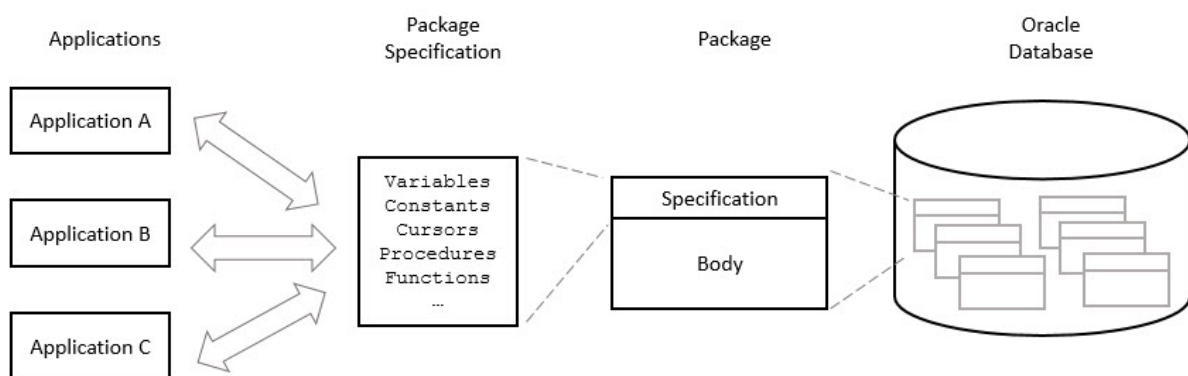
PL/SQL Package

A package is a way of logically storing the subprograms like procedures, functions, exception or cursor into a single common unit.

A package can be defined as an oracle object that is compiled and stored in the database. Once it is compiled and stored in the database it can be used by all the users of database who have executable permissions on Oracle database.

Typically, a package has a specification and a body. A package specification is mandatory while the package body can be required or optional, depending on the package specification.

The following picture illustrates PL/SQL package:



Benefits of using Package

Following are some of the benefits of packages in PL/SQL:

REUSABILITY

Whenever a package is created, it is compiled and stored in the database. So, you write the code once which can be reused by other applications.

OVERLOADING

Two or more procedures or functions can be created in a package with the same name.

CREATING MODULES

A large application can be created by simply creating modules(or subprograms) clearly defined and easy to work.

IMPROVES PERFORMANCE

Package code gets loaded inside the SGA(system global area) of Oracle at first call itself due to which other subsequent calls will work very fast.

GLOBAL DECLARATION

If the objects(procedures, functions, variables, constants, exceptions, cursor etc) are declared globally in a package, they can be easily used when required.

Components of Package

Package has two basic components:

- **Specification:** It is the declaration section of a Package
- **Body:** It is the definition section of a Package.

Package Specification

Package specification consists of a declaration of all the public variables, cursors, objects, procedures, functions, and exception.

Below are few characteristics of the Package specification.

- The elements which are all declared in the specification can be accessed from outside of the package. Such elements are known as a public element.
- The package specification is a standalone element that means it can exist alone without package body.
- Whenever a package has referred an instance of the package is created for that particular session.
- After the instance is created for a session, all the package elements that are initiated in that instance are valid until the end of the session.

Syntax for package specification:

```
CREATE [OR REPLACE] PACKAGE <package_name>

IS/AS
<sub_program and public element declaration>
.
.
END <package name>
```

Package Body

It consists of the definition of all the elements that are present in the package specification. It can also have a definition of elements that are not declared in the specification, these elements are called private elements and can be called only from inside the package.

Below are characteristics of a package body.

- It should contain definitions for all the subprograms/cursors that have been declared in the specification.
- It can also have more subprograms or other elements that are not declared in specification. These are called private elements.
- It is a dependable object, and it depends on package specification.
- The state of the package body becomes 'Invalid' whenever the specification is compiled. Therefore, it needs to be recompiled each time after the compilation of specification.
- The private elements should be defined first before they are used in the package body.
- The first part of the package is the global declaration part. This includes variables, cursors and private elements (forward declaration) that is visible to the entire package.
- The last part of the package is Package initialization part that executes one time whenever a package is referred first time in the session.

Syntax for package body:

```
CREATE [OR REPLACE] PACKAGE BODY <package_name>
IS
<global_declaration part>
<Private element definition>
<sub_program and public element definition>
.
<Package Initialization>
END <package_name>
```

Using the Package Elements

The package elements (variables, procedures or functions) are accessed with the following syntax –

package_name.element_name;

Example of creating and using a package:

Let's assume following **employee** table with data,

Id	Name	Salary
1	Ram	35000
2	Gita	45000
3	Hari	35000

Step 1: Create a package specification:

```
CREATE PACKAGE emp_sal AS
    PROCEDURE find_sal(eid employee.id%type);
END emp_sal;
/
```

After successful run, you'll get following output:

Package created.

Step 2: Create a package body:

```
CREATE OR REPLACE PACKAGE BODY emp_sal AS

    PROCEDURE find_sal(eid employee.id%TYPE)
    IS
        sal employee.salary%TYPE;
    BEGIN
        SELECT salary INTO sal
        FROM employee
        WHERE id = eid;
        dbms_output.put_line('Salary: ' || sal);
    END find_sal;
END emp_sal;
/
```

Output:

Package body created.

Step 3: Using package elements

```
DECLARE
    eid employee.eid%TYPE:=&eid;
BEGIN
    emp_sal.find_sal(eid);
END;
/
```

Output:

Enter value for eid: 2
Salary: 45000

PL/SQL procedure successfully completed.

Example – 2

Let's assume we have a **STUDENT** table as specified below:

ROLLNO	SNAME	AGE	COURSE
1	Ram	20	BIM
2	Shyam	21	CSIT
3	Hari	18	BCA
4	Sita	20	BCA
5	Gita	19	BBA

PL/SQL code for package specification:

```
CREATE OR REPLACE PACKAGE pkg_student IS
    PROCEDURE updateRecord(sno student.rollno%type);
    FUNCTION deleteRecord(snm student.sname%type)
        RETURN boolean;
END pkg_student;
```

PL/SQL code for package body:

```
CREATE OR REPLACE PACKAGE BODY pkg_student IS
    PROCEDURE updateRecord(sno student.rollno%type)
    IS
        BEGIN
            Update student set age=23 where rollno=sno;
            IF SQL%FOUND THEN
                dbms_output.put_line('RECORD UPDATED');
            ELSE
                dbms_output.put_line('RECORD NOT FOUND');
            END IF;
        END updateRecord;

    FUNCTION deleteRecord(snm student.sname%type) RETURN boolean
    IS
        BEGIN
            Delete from student where sname=snm;
            RETURN SQL%FOUND;
        END deleteRecord;
END pkg_student;
```

PL/SQL code for calling the Procedure and Function used in Package.

```
DECLARE
    sno student.rollno%type;
    s_age student.age%type;
    snm student.sname%type;
BEGIN
    sno := &sno;
    snm := &snm
    pkg_student.updateRecord(sno);
    IF pkg_student.deleteRecord(snm) THEN
        dbms_output.put_line('RECORD DELETED');
    ELSE
        dbms_output.put_line('RECORD NOT FOUND');
    END IF;
END;
```

Output:

```
Enter value for sno: 2
Enter value for snm: Hari
RECORD UPDATED
RECORD DELETED
PL/SQL procedure successfully completed.
```

After selecting student table we get following records,

ROLLNO	SNAME	AGE	COURSE
1	Ram	20	BIM
2	Shyam	23	CSIT
4	Sita	20	BCA
5	Gita	19	BBA